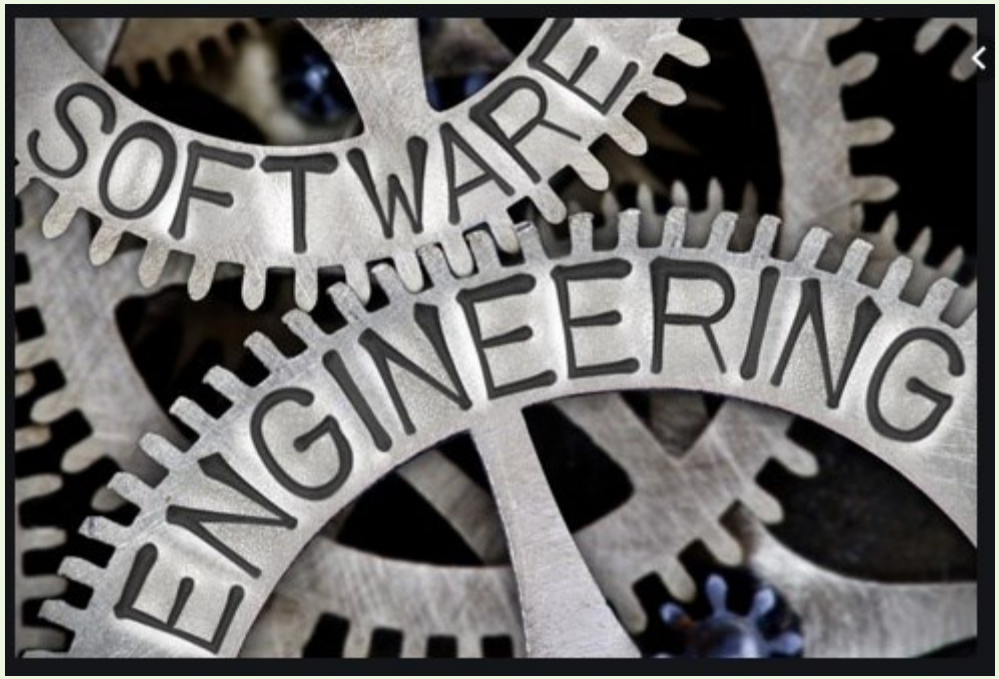
Software Engineering – Project Management

Module-2



Faculty :

Dr. Suchismita Rout

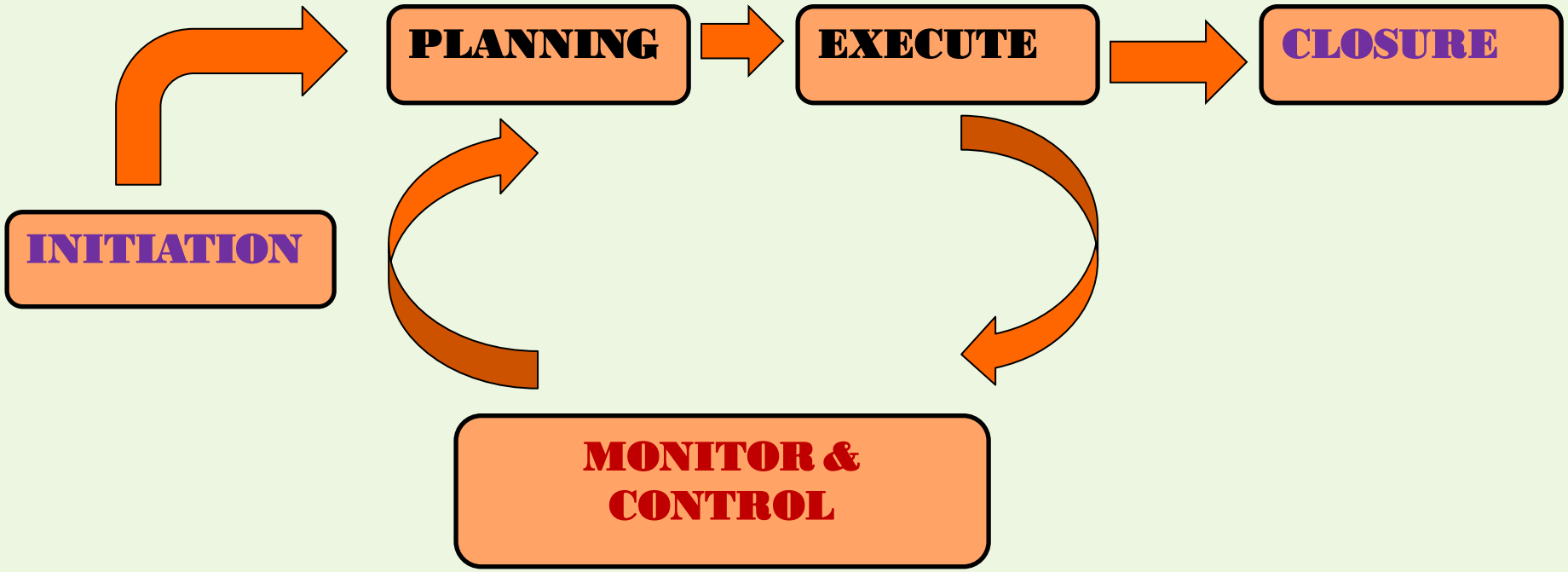
Associate Professor

Module2 Contents

- **Software Project Management - topics**
  - **Project Planning**
  - **Metrics for Project Size Estimation**
  - **Scheduling, Staffing**
  - **Project Estimation Techniques**
  - **COCOMO model**
  - **Halstead's Software Science**
  - **Risk Management**

Software Project Management

Project Management life cycle



- **Monitoring** : Compare Planned Progress with Actual Progress
- **Controlling** : Taking corrective action if required

Project planning

Project Planning in 10 Steps

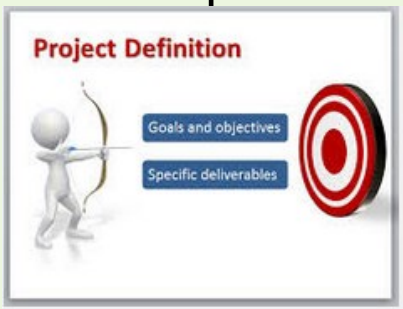
0. Select Project
1. Identify project scope & objectives
2. Identify project infrastructure
3. Analyze project characteristics & Select SDLC model
4. Identify project deliverables & activities
5. Estimate effort for each activity
6. Identify activity risks
7. Allocate Resources
8. Review/Publish plan
9. & 10. Execute Plan & Lower level planning

Step wise planning activities

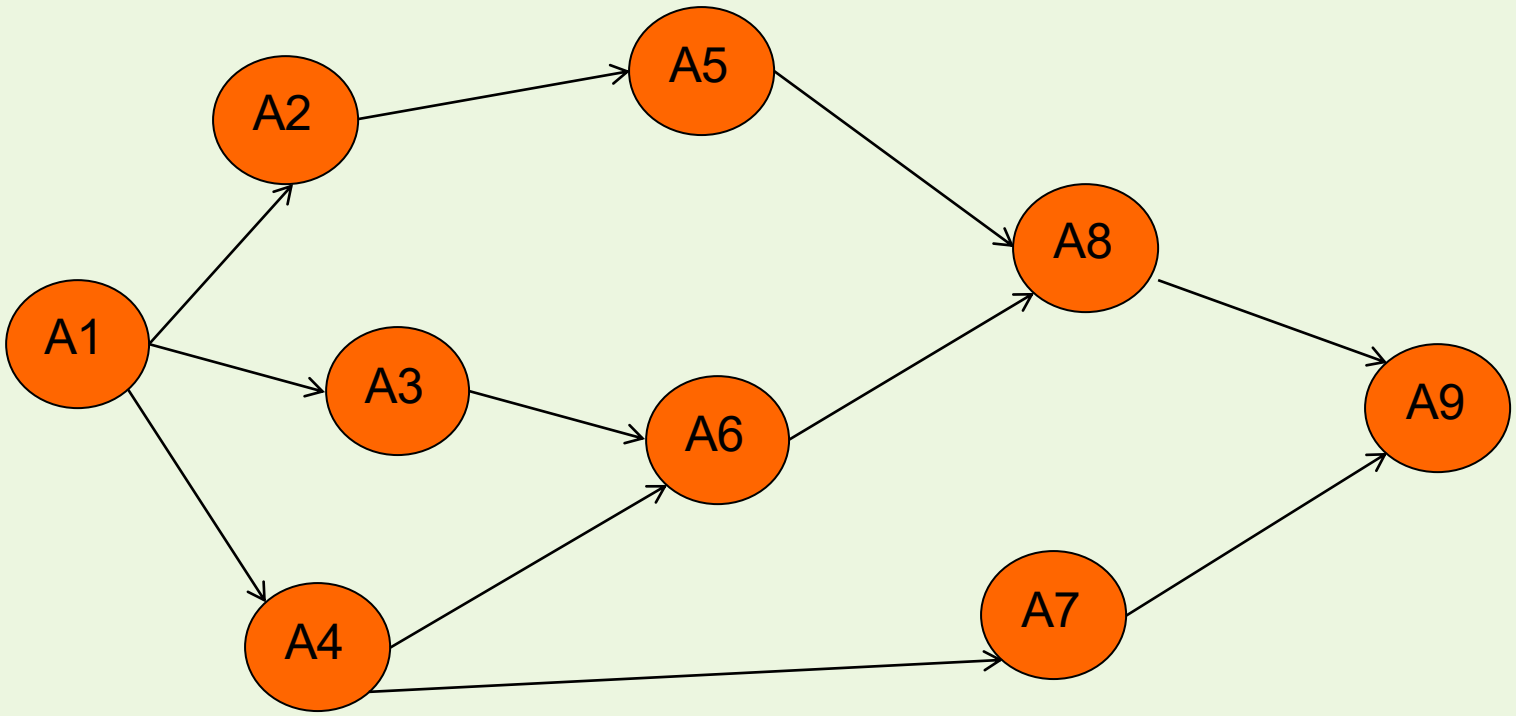
Step	Activities
0	**Select Project** (based on strategic evaluation)
1	**Identify project scope & objectives** (Have all parties agree to the objectives & commit to the success) Id. Objectives of the project Establish project authority (Project Manager) Id. all stakeholders (List stakeholders & their needs) Modify objectives based on stakeholder analysis Establish communication methods (Whom & How)
2	**Identify project infrastructure** (Existing inf. h/w, s/w, logistics) Establish relationship between project & strategic planning (Decide priority of the project) Id. Installation standards & procedures Id. Project team organization

Step wise planning activities

Step	Activities
3	**Analyze project characteristics** (for selecting appropriate methods) Distinguish the project (Objective driven or Product driven) Analyze other characteristics & . High level risks Select **lifecycle approach** Review overall resource estimates
4	**Identify project deliverables & activities** Id. deliverables (SRS, Set of components, test plans/results) Document generic product flows (SRS → Design Module → Develop Module → Test module, WBS) Recognize product components Produce **activity network** Take into account(modify) stages & check points



Activity Network



Step wise planning activities

Step	Activities
5	**Estimate effort for each activity** Carry out bottom-up estimates Revise Plan to create controllable activities (long activities to be broken down into smaller subtasks)
6	**Identify activity risks** Id. & quantify activity based risks (Id. risks, estimate - **probability & impact**) Plan for countering the risks Adjust overall plan to account for risks
7	**Allocate resources** Id & allocate resources Revise Plan/estimates based on resource constraints

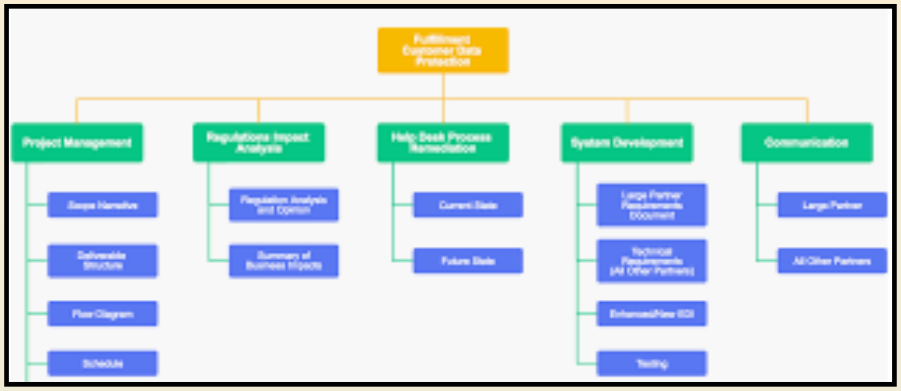
Step wise planning activities

Step	Activities
8	**Review/Publish plan** Review quality aspects of the Project Plan Document Plans & obtain agreement
9 & 10	**Execute Plan & Lower level planning** Implement the documented plan Plan in detail as you get more info.

✓ **Project Management is a Iterative process & Project Plan is updated many times during a project**

S/W Project Management Plan (SPMP)

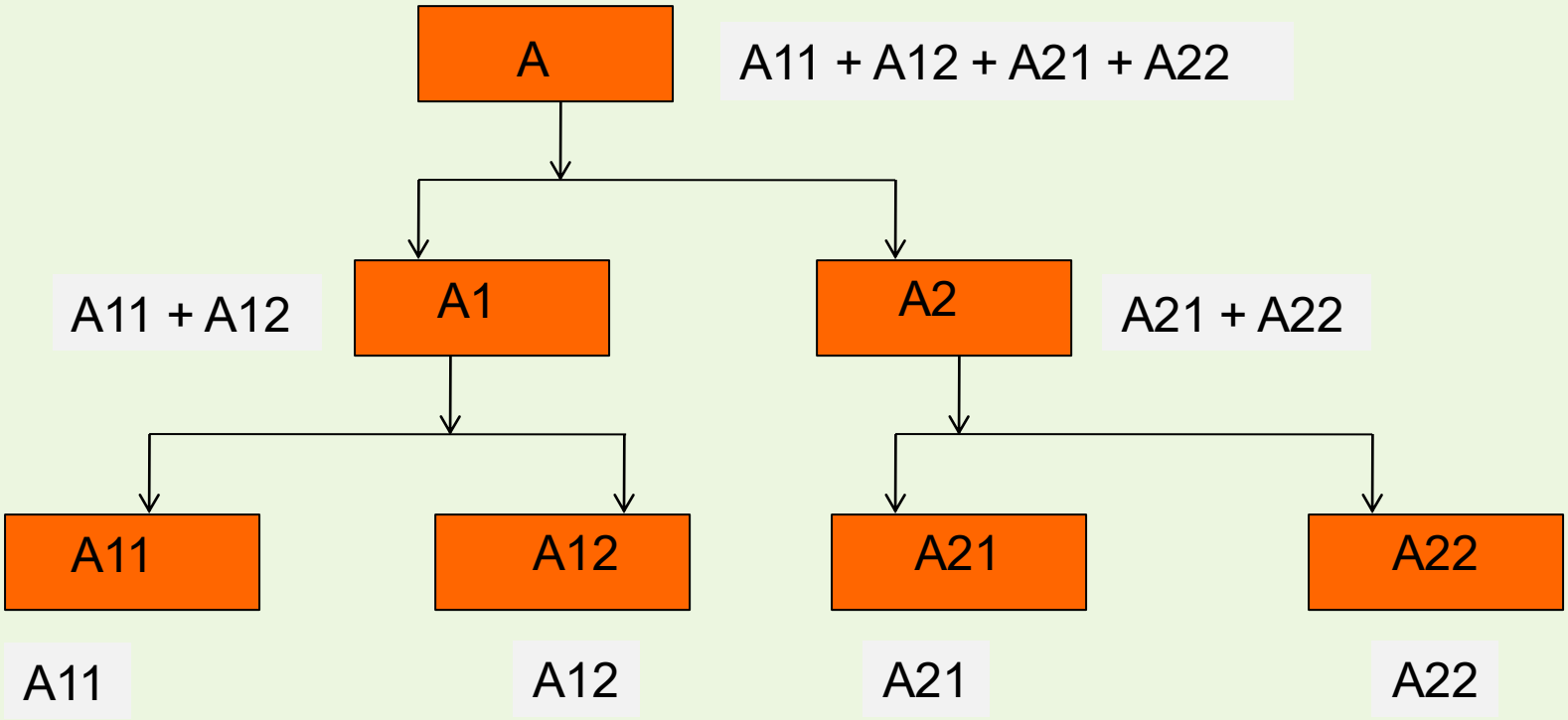
- **WBS** *(Work Breakdown Structure)*
  - Decomposition of project into smaller manageable parts – WBS Units



- **WBS Units**
  - Each WBS unit must have a *Clear Outcome, Accountability, is Trackable, has Well defined interface*

- **"WBS"** is created:
  - To Divide & Conquer & to estimate accurately
- **How WBS is done**
  - Id. Logical components & minimize dependency between WBS Units
- This is followed by "**Bottom-Up**" estimation approach

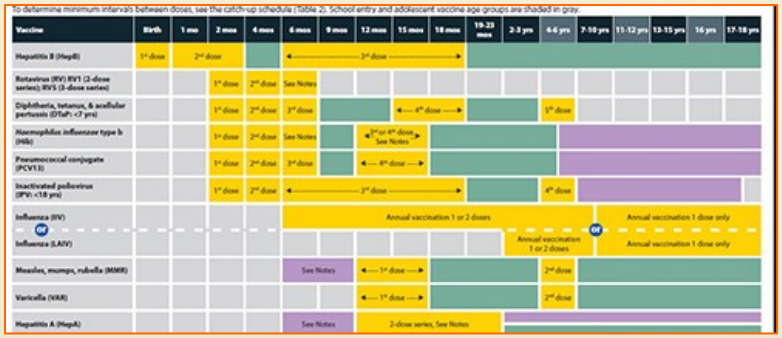
Bottom-up Estimate



- **SPMP includes:**
  - Size estimates *(KLOC, FPs…)*
  - Effort estimates
  - Cost estimates
  - Internal & External Milestones
  - Resource & Work allocation
  - Project schedule
  - Communication & Status Reporting plan
  - Risk management plan

Project Schedule Estimation

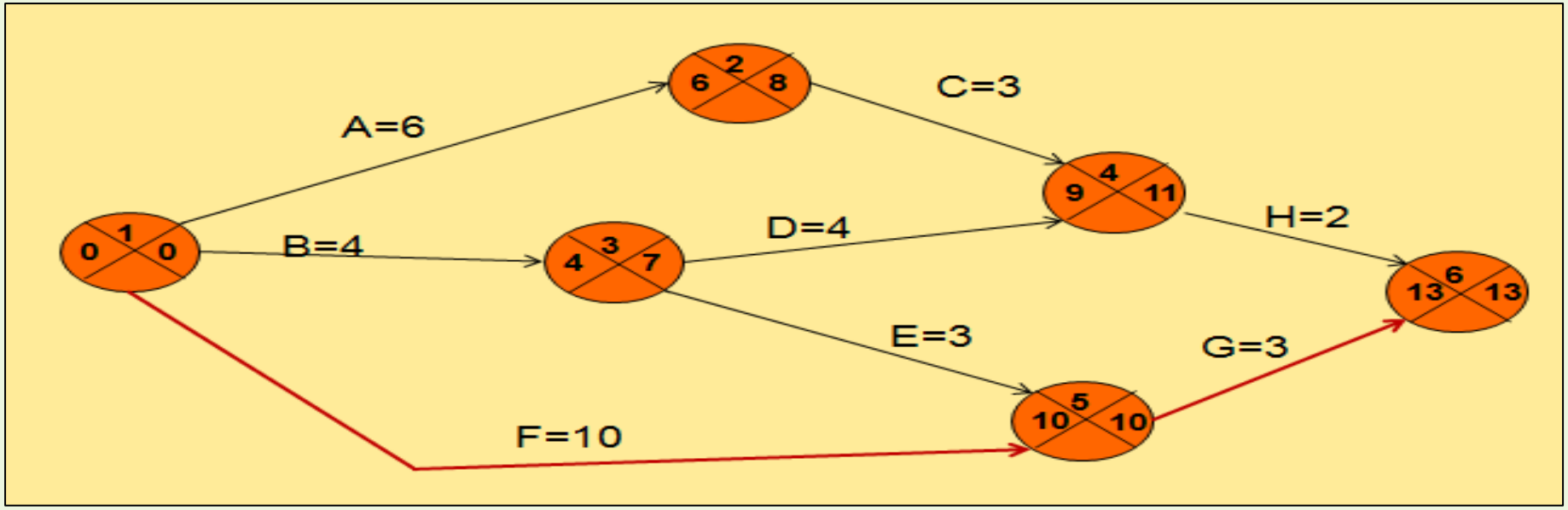
- **"Calender Time"** to deliver
- Provides answer to **"When can we deliver what"**
- **Schedule estimation** is done taking into account :
  - *Deadline limits*
  - *Resources available*
  - *H/W Constraints*
  - *Internal & External dependencies*



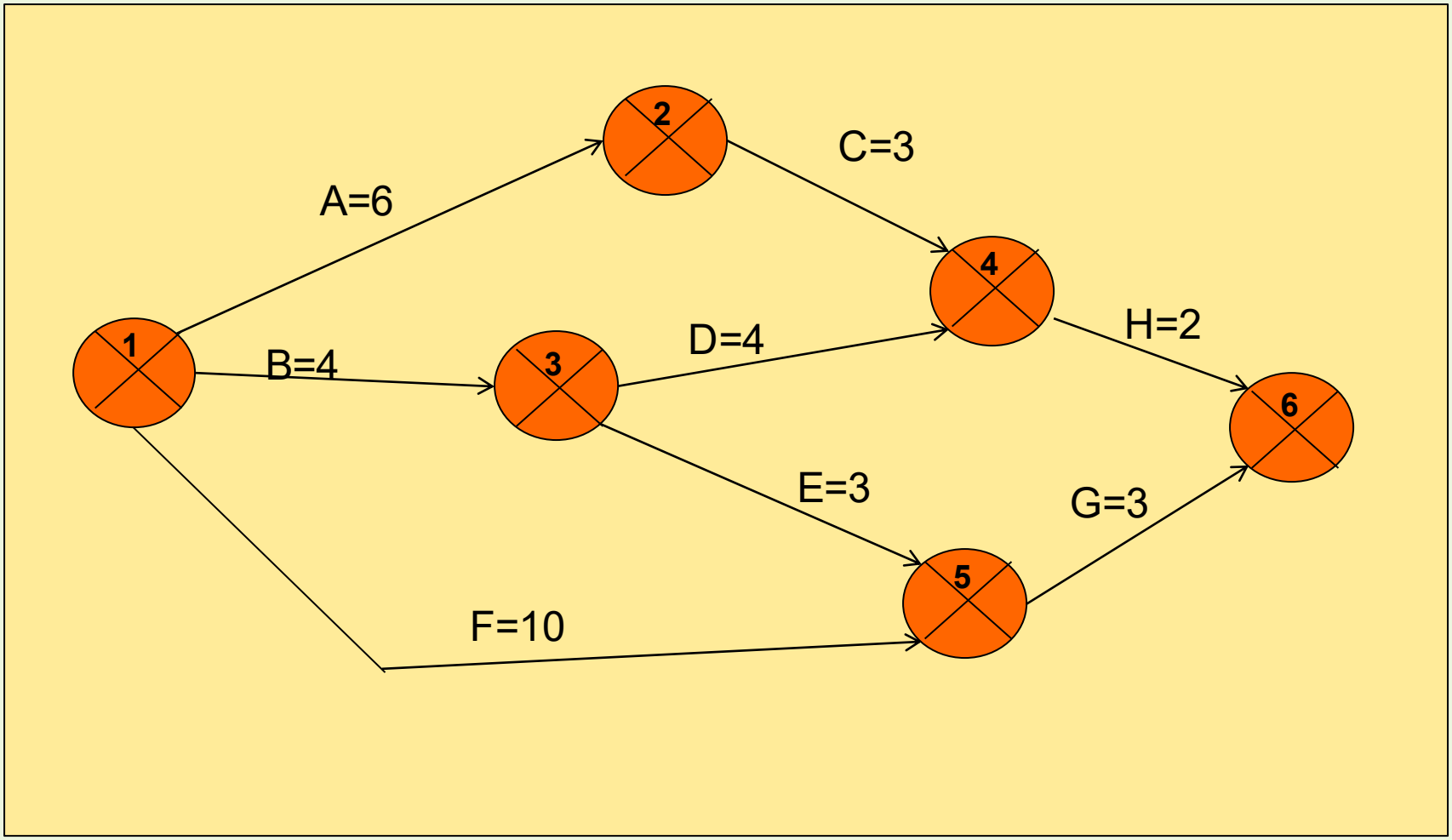
- Done by constructing "**Activity Network model**" & using **CPM (Critical Path Method)**

Activity network model & CPM (critical Path method)

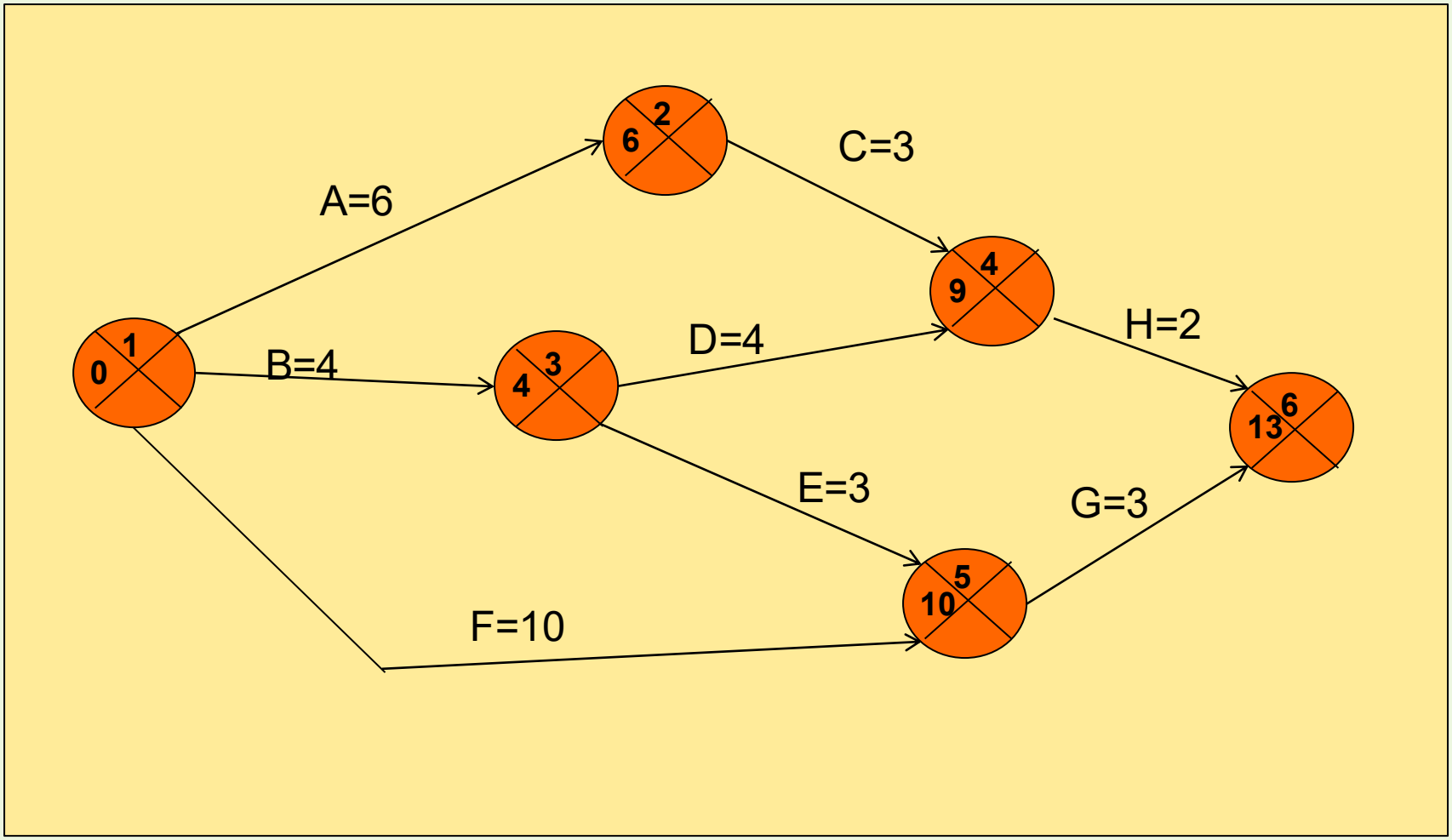
- Activities are "nodes"
- Durations are "edges"
- Direction of edges show "precedence / dependency" between activities



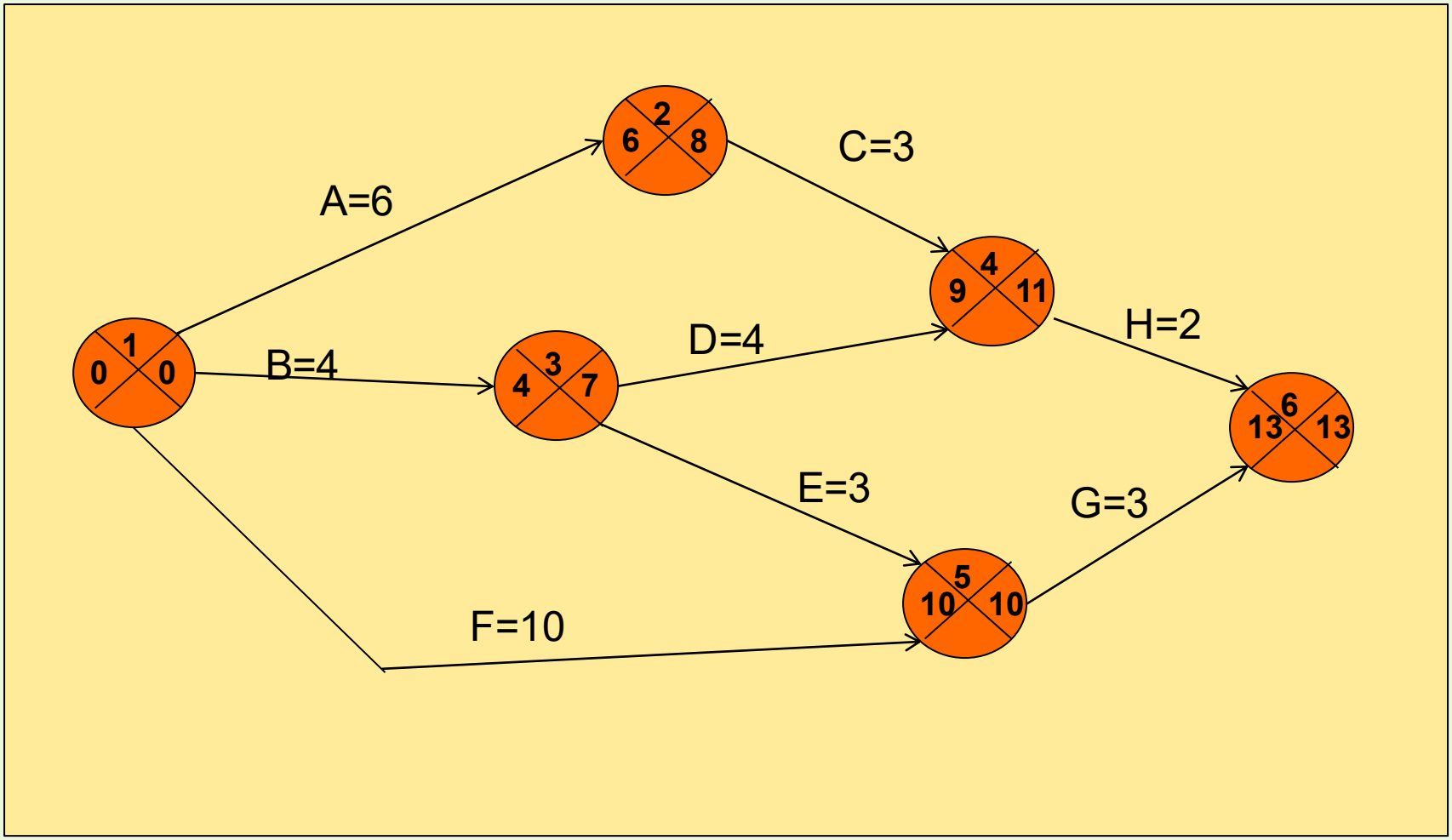
Scheduling of Activities : Activity network model (CPM method)



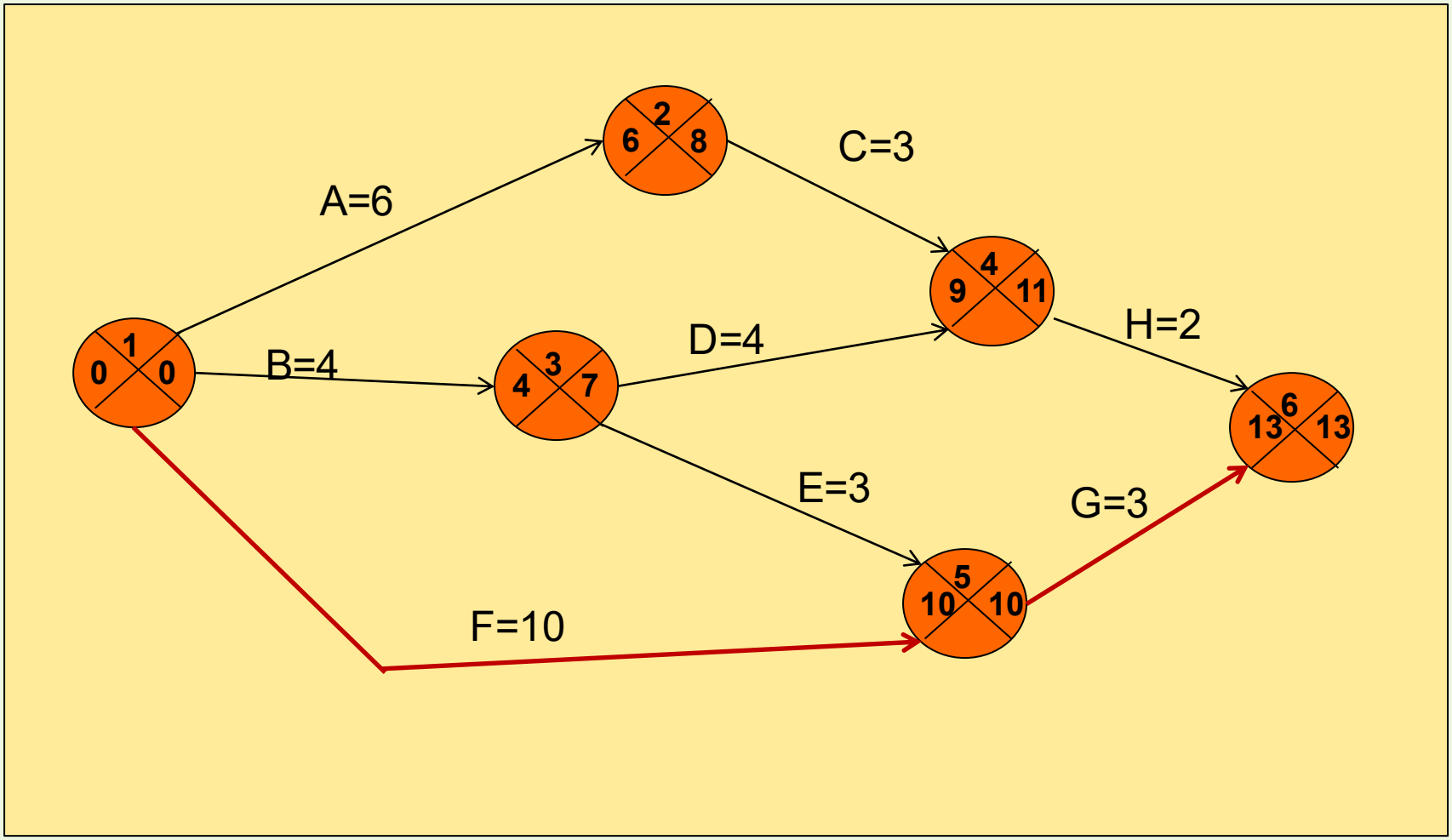
CPM n/w after forward pass



CPM n/w after backward pass



Critical Path



Critical Path

- **Difference** (also called **Slack** or **Float)**

  latest start/completion date – earliest start/completion date

- **Events/Tasks** with **0 slack** are **Critical events**

- The **path joining critical events** is called "**Critical Path**"

Critical Path

- Need to **pay more attention** to events on critical path
  - To Avoid **delays**
- To shorten a project, the **critical path must be shortened**
  - By adding more resources/overtime

Project Estimation & COCOMO

Objectives

- **To understand different methods of "Estimating" projects**
- **COCOMO approach**

Introduction

A "Successful project" is the one that is

- Delivered on-time
- Within budget
- Is of required quality

Realistic estimates are critical to achieve the above objectives

Estimating s/w projects

Estimation is done during below stages :

- **Strategic Planning** (to prioritize the project)
- **Feasibility Study**
- **Requirement Specification**
- **Evaluation of supplier proposal**
- **Project Planning**

Basis of s/w estimating

Estimation is done based on below factors:

- Based on **Historical data**
  - Env. factors, S/w & Tools used, staff experience have to considered
- Based on **Measure of work**
  - Based on work content (like SLOC, KLOC)
  - Difficult to apply for app. building tools
- Based on **Complexity**
  - Two programs with same SLOC may take different times based on complexity (complexity needs to be taken into account)

S/W Effort estimating techniques

- **Algorithmic methods**
  - Uses characteristics of target system (**Ex**: size of system) & the implementation environment to predict effort
- **Empirical estimation techniques**
  - **Expert Judgment** (Take expert advise to estimate)
  - **Delphi Technique**
- **Analogy** (Estimate based on similar completed projects)
- **Parkinson** (Estimate Based on staff effort available)
- **Price to win** (Estimate to win the contract)
- **Top-Down** (Overall effort is distributed to tasks)
  - **Ex**: Building a house based on sq. ft rate
- **Bottom-up** (Effort of individual tasks are summed up)

Bottom-up VS Top-down estimation

Bottom-up estimation

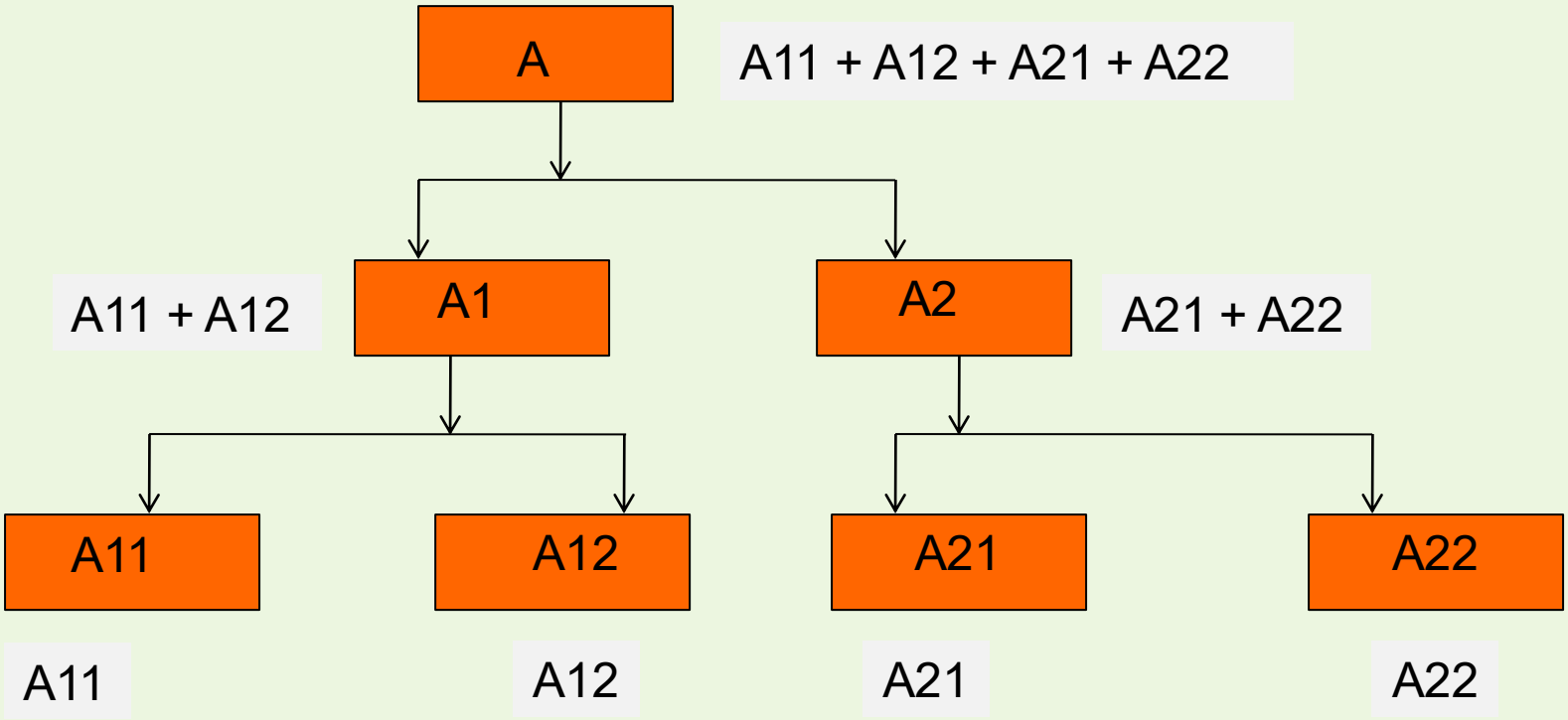
- Breaks the project up to tasks level
- Called WBS *(work breakdown structure)*
- Tasks can be executed by a single resource
- Estimate effort for each task
- Sum up to find total estimate

Top-down estimation

- Parametric model used to estimate the effort based on characteristics of the system & distribute to task level
- Building a house (estimation based on sq. feet of area)
  - Estimation based on a characteristics of the system
  - **Effort = System size X Productivity**

    Ex: 50 (Kloc) X 40 (man-days per Kloc) = 2000 man-days

Bottom-up Estimate



Expert Judgement & Estimation by Analogy

✓ **Expert Judgment**

- People very knowledgeable about the s/w & type of project can provide the estimate based on experience & expertise

✓ **Estimation by Analogy**

- Case-based reasoning
- Estimation based on similar completed projects
- Adjustments made based on the differences

Delphi Method of estimation

- The **Delphi method** is a process used to arrive at a **group consensus** (agreement) on **estimation** given a **panel of experts**

- **Experts** respond to **questionnaires**

- Their **responses** are **aggregated** and **shared**

- In case of **differences**, further **rounds of discussions** take place to **arrive at an agreement**

COCOMO Model

- ✓ **COCOMO** (**COnstructive COst MOdel**) proposed by Boehm

- ✓ Divides **Software Projects** / **Products** into **Three classes** based on **development complexities**:

  1. **Organic**
  2. **Semidetached**
  3. **Embedded**

Elaboration of Product classes

1. **Organic:**
   - ✓ Relatively **small project**
   - ✓ **Team Experienced** with similar projects
   - ✓ **Well-understood** applications

2. **Semidetached:**
   - ✓ **Project team** consists of a **mixture of experienced** & **inexperienced** staff
   - ✓ **Limited experience** on similar systems

3. **Embedded:**
   - ✓ **Complex** system
   - ✓ The software is strongly **coupled to complex hardware**
   - ✓ **Real-time systems**

COCOMO Model (Cont.)

- ✓ For each of the three product categories:
  - ✓ **Boehm** provides **equations** to predict:

    - ✓ **Effort** in person-months
    - ✓ **Project duration** in months

    - ✓ One **PM** is included in the effort
    - ✓ Takes into account **productivity loss** due to holidays, weekly offs...

COCOMO Model (cont.)

- ✓ According to Boehm, Software cost estimation is done through three stages:

  1. **Basic COCOMO**
  2. **Intermediate COCOMO**
  3. **Complete COCOMO**

(1) Basic COCOMO Model

- ✓ Gives only an approximate estimation:

  - ✓ **Effort = a1*(KLOC)^a2 PM**
  - ✓ **Tdev = b1*(Effort)^b2 Months**

  - ✓ **KLOC –** Estimated code in kloc
  - ✓ **a1,a2,b1,b2** - Constants for different categories of software products (Organic, Semi-detached, Emb.)
  - ✓ **Tdev** - Estimated time duration to develop the software in months

Development Effort & Time Estimation

- **Values of a1,a2 & b1,b2**
  - ✓ **Organic** :
    - ✓ Effort = 2.4*(KLOC)^1.05 PM
    - ✓Tdev = 2.5 (Effort)^0.38 Months
  - ✓ **Semi-detached**:
    - ✓Effort = 3.0*(KLOC)^1.12 PM
    - ✓Tdev = 2.5 (Effort)^0.35 Months
  - ✓ **Embedded**:
    - ✓Effort = 3.6*(KLOC)^1.20PM
    - ✓Tdev = 2.5 (Effort)^0.32 Months

(2) Intermediate COCOMO

- ✓ **Basic COCOMO** model assumes
  - ✓ Effort and development time depend on **product size alone**

- ✓ Several other parameters affect effort and development time:

  EX:

  - ✓ Reliability of requirements
  - ✓ Availability modern tools
  - ✓ Size of data to be handled ...

Intermediate COCOMO

- ✓ **Intermediate COCOMO** model:
  - ✓ Refines the initial estimate obtained by the basic COCOMO
    - ✓ by using a set of **15 cost drivers (multipliers)**
  - ✓ Multiply cost driver values with the estimate obtained using the basic COCOMO

✓ **Effort = [Effort $_{Basic_cocomo}$] * [CD_1 * CD_2 … CD_{15}] PM**

Cost Drivers

Cost Drivers	Ratings				
	Very Low	Low	Nominal	High	Very High
Product attributes					
Required software reliability	0.75	0.88	1.00	1.15	1.40
Size of application database		0.94	1.00	1.08	1.16
Complexity of the product	0.70	0.85	1.00	1.15	1.30
Hardware attributes					
Run-time performance constraints			1.00	1.11	1.30
Memory constraints			1.00	1.06	1.21
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30
Required turnabout time		0.87	1.00	1.07	1.15
Personnel attributes					
Analyst capability	1.46	1.19	1.00	0.86	0.71
Applications experience	1.29	1.13	1.00	0.91	0.82
Software engineer capability	1.42	1.17	1.00	0.86	0.70
Virtual machine experience	1.21	1.10	1.00	0.90	
Programming language experience	1.14	1.07	1.00	0.95	
Project attributes					
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82
Use of software tools	1.24	1.10	1.00	0.91	0.83
Required development schedule	1.23	1.08	1.00	1.04	1.10

Shortcoming of basic and intermediate COCOMO models

In Both models:

- ✓Consider a software product as a **single homogeneous entity**
- ✓However, most large systems are made up of several **smaller sub-systems**
  - ✓Some sub-systems may be organic type, some may be semi-detached or embedded

(3) Complete COCOMO

- ✓ Each **sub-system** may be of **different types – organic, semi-detached or embedded**
- ✓ **Cost of each sub-system** is **estimated separately**
- ✓ Costs of the sub-systems are **added** **to obtain total cost**
- ✓ This **Reduces** the **margin of error** in the final estimate

Example

- ✓ A **Management Information System (MIS)** for an organization with many branches has:

  1. **Database part** (semi-detached)
  2. **Graphical User Interface (GUI) part** (organic)
  3. **Communication part** (embedded)

- ✓ Costs of the components are <u>estimated separately</u>:

  - ✓ <u>Summed</u> up to give the <u>overall cost of the system</u>

Halstead's Software Science

- **Halstead complexity measures** are software metrics introduced by *M.H Halstead* in *1977*
- Consists of a number of metrics to measure **complexity of code** based on it's *algorithm*
- These metrics are computed statistically from the **code**
- Halstead's goal was to identify measurable properties of software, and the relations between them

Metrics Calculation steps

For a given problem, Let:

- η_1 = the number of distinct operators
- η_2 = the number of distinct operands
- N_1 = the total number of operators
- N_2 = the total number of operands

From these numbers, several measures can be calculated:

- Program vocabulary: $\eta = \eta_1 + \eta_2$
- Program length: $N = N_1 + N_2$
- Calculated estimated program length: $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$
- Volume: $V = N \times \log_2 \eta$
- Difficulty : $D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$
- Effort: $E = D \times V$

Metrics Calculation steps

The difficulty measure is related to the difficulty of the program to write or understand

The effort measure translates into actual coding time using the following relation,

- Time required to program: $T = \frac{E}{18}$ seconds

Halstead's delivered bugs (B) is an estimate for the number of errors in the implementation.

- Number of delivered bugs : $B = \frac{E^{\frac{2}{3}}}{3000}$ or, more recently, $B = \frac{V}{3000}$ is accepted

Example

```c
main()
{
    int a, b, c, avg;
    scanf("%d %d %d", &a, &b, &c);
    avg = (a + b + c) / 3;
    printf("avg = %d", avg);
}
```

The unique operators are: `main`, `()`, `{}`, `int`, `scanf`, `&`, `=`, `+`, `/`, `printf`, `,`, `;`

The unique operands are: `a`, `b`, `c`, `avg`, `"%d %d %d"`, `3`, `"avg = %d"`

- $\eta_1 = 12, \eta_2 = 7, \eta = 19$
- $N_1 = 27, N_2 = 15, N = 42$
- Calculated Estimated Program Length: $\hat{N} = 12 \times log_2 12 + 7 \times log_2 7 = 62.67$
- Volume: $V = 42 \times log_2 19 = 178.4$
- Difficulty: $D = \frac{12}{2} \times \frac{15}{7} = 12.85$
- Effort: $E = 12.85 \times 178.4 = 2292.44$
- Time required to program: $T = \frac{2292.44}{18} = 127.357$ seconds
- Number of delivered bugs: $B = \frac{2292.44^{\frac{2}{3}}}{3000} = 0.05$

Risk Management

Identify Risks

Objective:

- To **identify** the **Risks** that may cause a **project** to complete late or go over budget

Identify 3 types of risks :

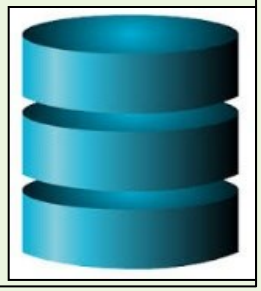
1. Risks caused by **Estimation difficulties**
2. Risks caused by **Assumptions** made during **planning**
3. Risks caused by **unforeseen events**

1. Estimation Difficulties

✓ Some tasks are harder to estimate than others:

- Due to lack of experience of the **estimator** OR
- Due to nature of the task

- **Ex:**
  - Debugging is harder to estimate than creating user manuals



✓ Estimation can be improved by analyzing historical data

2. Assumptions

VALIDATE
YOUR
ASSUMPTIONS

- If assumptions made during planning are invalid,
  - Project will be at risk

Ex: Wrong selection of life cycle model or design approach

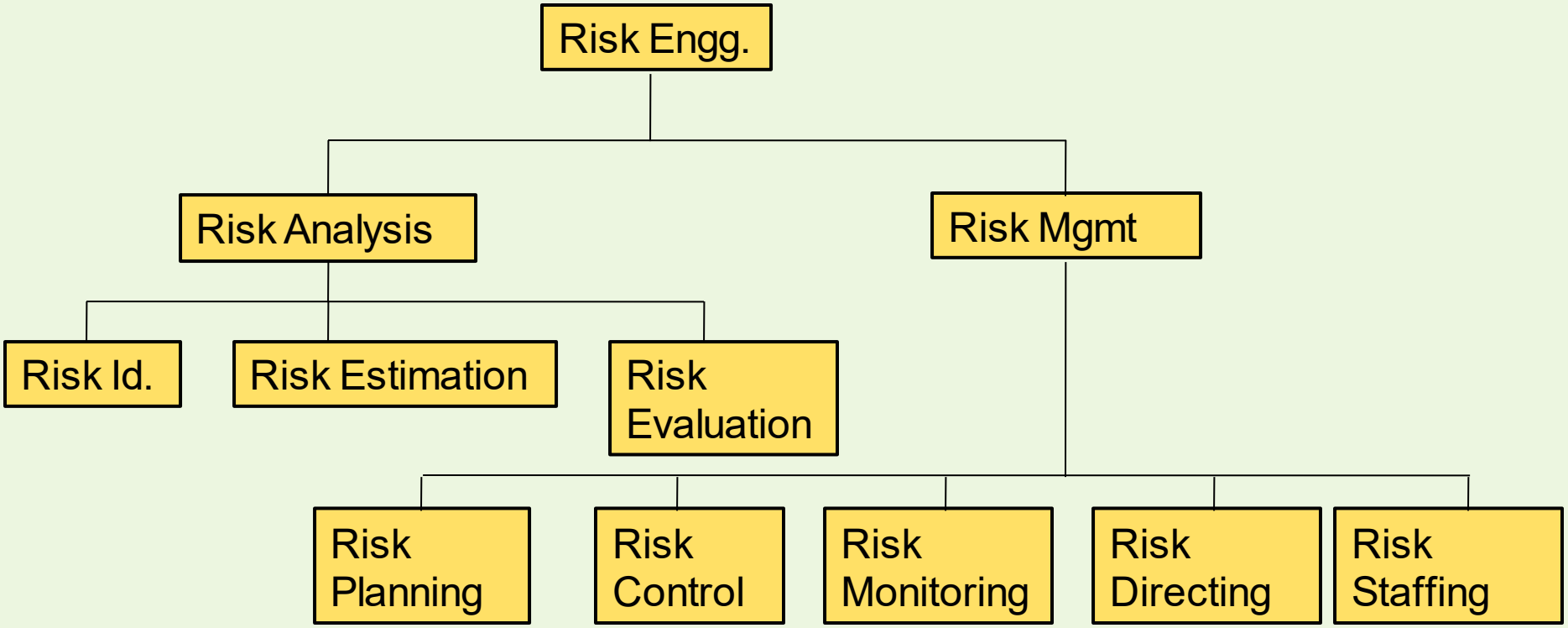
3. Unforeseen Events

- There may be some rare unforeseen event happening
  - **Ex**: Tech. used becomes unstable / unsupported,
    Drastic rule changes

Risk Engineering structure

Objectives:

- To avoid or minimize effects of project risks



Managing Risks

Risk Analysis:

- ✓ **Risk Identification** : List all the risks affecting the project
- ✓ **Risk Estimation** : Assessing the likelihood & impact of each risk
- ✓ **Risk Evaluation** : Ranking the risks

Risk Management:

- ✓ **Risk Planning** : Making 'contingency plans' & adding tasks to project. May need a risk manager
- ✓ **Risk Control** : Minimizing & Reacting to risks throughout the project
- ✓ **Risk Monitoring** : On-going task. Risk likelihoods & impacts may change
- ✓ **Risk Staffing** : Allocate Staff to manage risks in day-to-day basis

Risks Identification

- ✓ If a risk affects activities on '**critical path**' it will cause **delay**
- ✓ Use **checklist** to Identify Risks (2 types)
  - ✓ **Generic risks**
    - ✓ **Ex:** *Misunderstanding of requirements, Key person ill*
  - ✓ **Specific risks** (for a given project) – More difficult to id.
    - ✓ **Ex:** *Unknown techs used in the project, Safety-critical*

1. Risk Analysis:
1.2. Risks Estimation

- Determine "**Likelihood**" & "**impact**" (in monetary terms) of risks
- Calculate '**Risk Value"** (Expected cost)

Risk Exposure = Risk likelihood X Risk Impact

- Use "Risk Exposure" to **prioritize** or **rank** risks
- Risk scoring can be expressed as **(H, M, L)** or in scale of **1-10**

Risk Matrix

A **risk matrix** is a matrix that is used during risk assessment to define the level of risk by considering :

- The "**category of probability or likelihood**" against the "**category of consequence severity or impact**"

- This is a simple mechanism to increase visibility of risks & assist management decision making

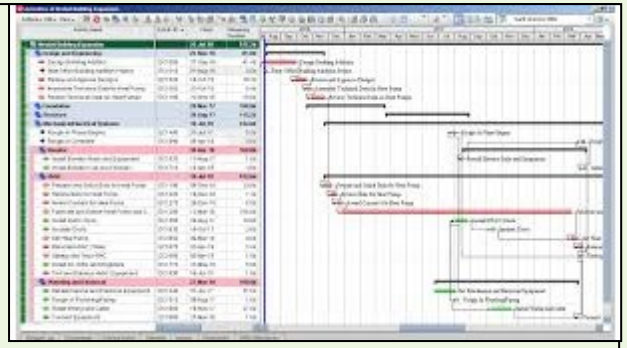
Probability \ Impact	Low	Medium	High
High	Low	Medium	High
Medium	Low	Medium	Medium
Low	Low	Low	Low

Strategies to Reduce Risks

5 Strategies are there :

- **Risk Prevention** *(Scheduling critical meetings **early**)*
- **Likelihood Reduction** *(Prototyping reduces chance of freq. reqt. changes)*
- **Risk Avoidance** *(Reduce functionality, Increase duration)*
- **Risk Transfer** *(Contracting, Insurance)*
- **Contingency Planning** *(Plan to minimize impact of remaining risks)*

1. Risk Analysis:
1.3. Risks Evaluation – on schedule



- All risks can not be eliminated
- Some will delay the project
- Id. such risks & their impact on duration
- **Can use 2 methods:**

1. **PERT**

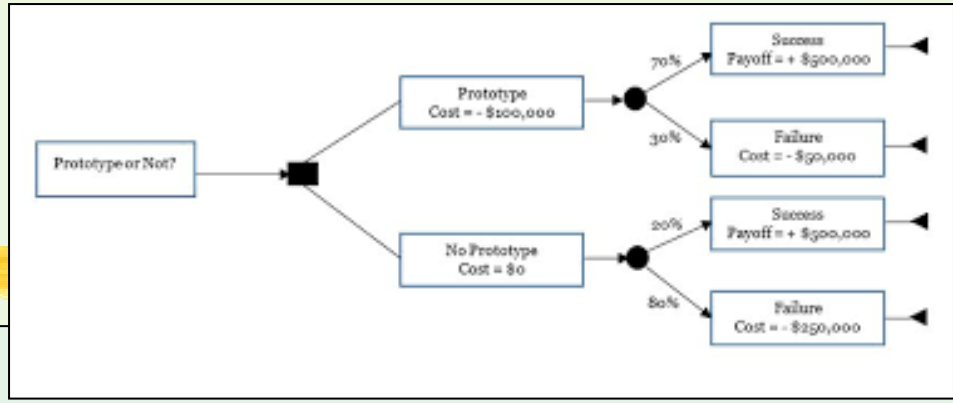
   - Uses 3 estimates : Most likely (m), Optimistic (a) & Pessimistic time (b)
   - PERT combines these 3 estimates into a single estimate

$$t_e = \frac{(a + 4m + b)}{6}$$

2. Using expected durations

- The event dates are not the 'earliest possible dates'
  - But 'expected dates'

Risk Evaluation

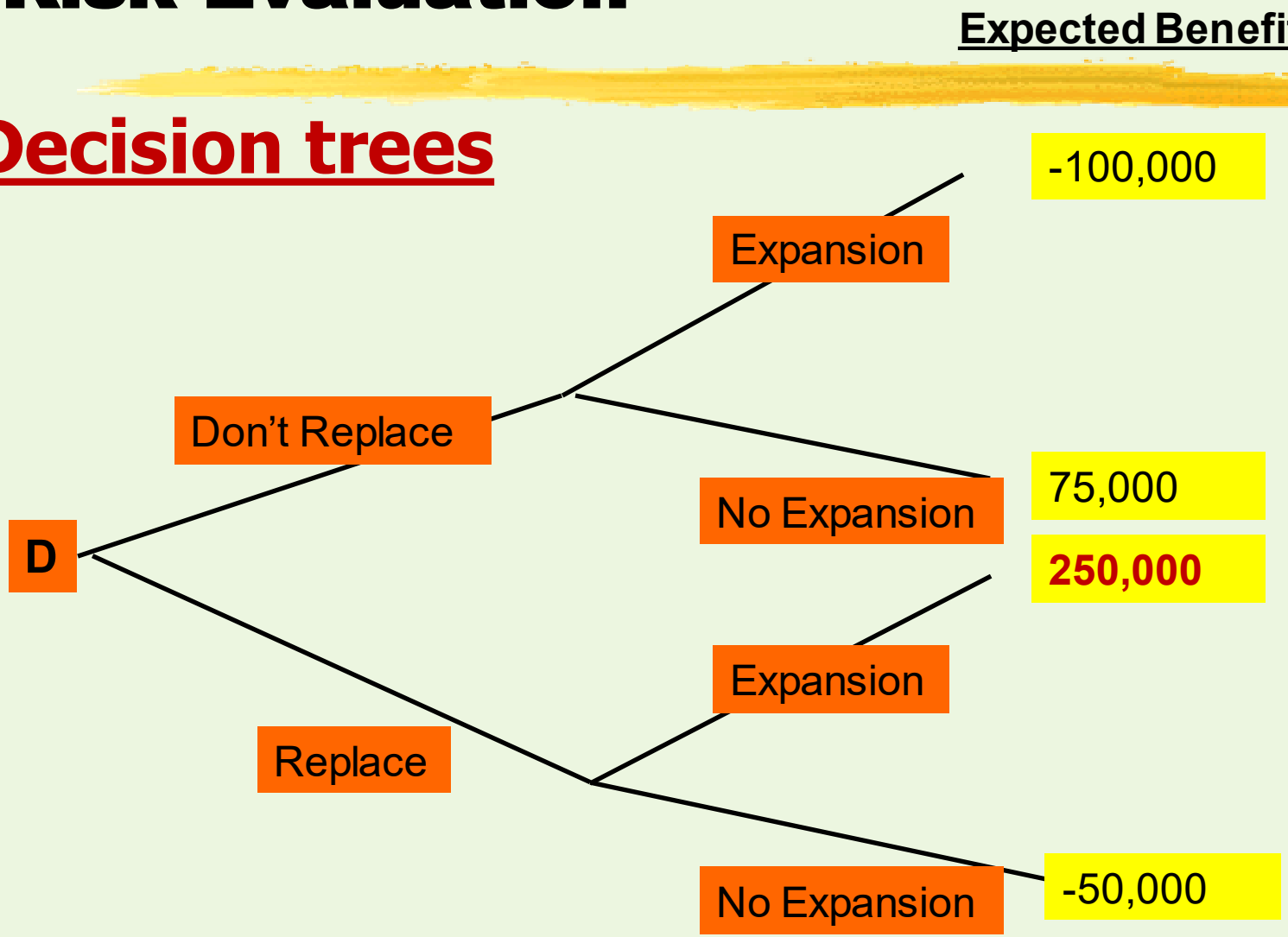


(2) Using "Decision Tree Analysis"

- It is important to see how a **decision** will affect the **future profitability** of a project
- **Ex:**
  - **Not replacing old systems** with **new ones** could result in **revenue loss**
  - **Replacing now** will result in **more spending**
- **D.T analysis** consists of evaluating **expected benefits** of taking **each path** from **a decision point**

Risk Evaluation

Decision trees



Thank You !!!